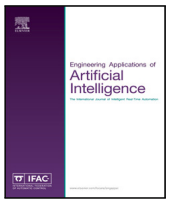
Engineering Applications of Artificial Intelligence 161 (2025) 112250



Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai



Research paper



Unmanned surface vehicle autonomous racing and obstacle avoidance with robust adversarial deep reinforcement learning

Jingyu Liu [a], Yuanda Wang [a,*], Changyin Sun [a,b]

[a] School of Automation, Southeast University, Nanjing, 210096, Jiangsu, China
[b] School of Artificial Intelligence, Anhui University, Hefei, 230601, Anhui, China

ARTICLE INFO

Keywords:
Adversarial deep reinforcement learning
Autonomous racing
Obstacle avoidance
Unmanned surface vehicles

ABSTRACT

This paper presents an autonomous racing control method for Unmanned Surface Vehicles (USVs) based on robust adversarial deep reinforcement learning (ADRL) algorithm, which leverages the strengths of both deep reinforcement learning and adversarial training. Adversarial obstacles and various tracks are employed to train a policy, so the proposed method can enhance the robustness and generalization of autonomous USV racing while ensuring effective obstacle avoidance. A simulation environment for USV racing was developed to conduct the experiments with Unity3D. The performance of the proposed method in handling diverse track scenarios and obstacle avoidance is demonstrated through simulations. Quantitative results show that ADRL achieves dramatic improvements over baseline DRL methods: collision rates are reduced by 99.85%, task completion rates are improved by 73.75%, and navigation time efficiency is enhanced by approximately 3% while maintaining superior safety performance.

1. Introduction

Unmanned systems (Fahlstrom et al., 2022) are a series of autonomous robotic devices capable of self-control without human intervention. These systems include various types, such as Unmanned Aerial Vehicles (UAVs) (Metekia et al., 2025), Unmanned Ground Vehicles (UGVs) (Farella et al., 2024), and Unmanned Surface Vehicles (USVs) (Chen et al., 2022; Gonzalez-Garcia and Castañeda, 2021). USVs are autonomous platforms that navigate on the water surface and represent an important branch of marine robotics. Due to the harsh marine environment and the unique characteristics of USV motion models (such as high nonlinearity, strong time-delay, and time-variability), the control and navigation tasks of USVs are significantly challenging (Barrera et al., 2021).

The increasing applications of USVs in both civilian and military fields highlight their growing significance, owing to their cost-effectiveness, adaptability, and ability to perform in complex environments (Chen et al., 2021; Bae and Hong, 2023). They are extensively employed in areas such as sea patrol, oceanography, water quality monitoring, and communication relays. Among these applications, rescue and pursuit at sea are emerged as a key focus, where the ability to swiftly navigate to a target while avoiding obstacles is crucial for USV. This type of requirement can be abstracted as a USV racing problem.

USV racing presents unique challenges that necessitate advanced navigation algorithms capable of dealing with dynamic environments and unpredictable obstacles. The primary task is not only to reach the destination quickly but also to ensure safe maneuvering around obstacles, which requires a balance between speed and collision avoidance. Over the years, significant strides have been made in developing algorithms aimed at enhancing USV navigation and control.

Navigation is divided into global navigation (Zheng et al., 2024) and local navigation (Mohanty et al., 2021). Global navigation algorithms can obtain the global information to guide USV in completing missions from a known global map of the environment, such as A* (Hart et al., 1968; Ma et al., 2022) algorithm and Dijkstra (2022) and Zhu et al. (2021). However, a good pre-built map usually requires more time and resources, so global navigation cannot be applied on a large scale in the short term. Any inaccuracies or outdated information in the maps can lead to navigation errors. Moreover, global navigation systems often struggle to adapt to unexpected obstacles in certain environments. It also may make USV navigation unsafe such as colliding with obstacle and being unable to reach the target positions (Jiang et al., 2020). Wang et al. (2023) proposed the dynamic target artificial potential field method and an edge computing architecture for global path planning, the experiments show that USVs can achieve a higher success rate to finish missions with moving obstacles and lower computing delay. Nevertheless, global navigation still cannot compare to local navigation in environments with unknown information. Local

* Corresponding author.
E-mail address: wangyd@seu.edu.cn (Y. Wang).

https://doi.org/10.1016/j.engappai.2025.112250
Received 15 May 2025; Received in revised form 11 August 2025; Accepted 5 September 2025
Available online 16 September 2025
0952-1976/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

navigation differs from global navigation in that the environmental information encountered is partially or completely unknown. Local navigation focuses on real-time decision-making based on immediate surroundings, often using sensors to avoid obstacles and navigate through dynamic environments. A method, Autonomous Navigation and Obstacle Avoidance (ANOA) based on deep reinforcement learning (DRL), is proposed for the autonomous navigation and obstacle avoidance of USVs (Wu et al., 2020). ANOA performs well in both static and dynamic environments. Furthermore, to enhance the effectiveness of multiple collision avoidance strategies for USVs, Fan et al. (2023) designed an improved methodology resulting in a highly efficient sampling, exploration, and learning process. Additionally, a novel dynamic area restriction technique is proposed to reduce computational expenses in this research.

The fundamental requirement for USV navigation algorithms is the corresponding control algorithms. With the advancement of research in various areas of USV, many control algorithms have been applied to USV-related fields, including sliding mode control (Cai et al., 2021), adaptive control (Mohammed et al., 2025), model predictive control (Zhao et al., 2021; Yan et al., 2020), backstepping control (Liu et al., 2017; Xie et al., 2020). Especially, reinforcement learning (RL) methods have been widely studied in the field of USV research in recent years (Wang et al., 2024a; Shimizu et al., 2022). RL algorithms outperform traditional algorithms in dynamic environment adaptation, handling complex tasks, automatic optimization, and strong generalization capabilities. Some corresponding research has been discussed above.

The local navigation algorithms based on RL above can make USV reach the target point while avoiding obstacles. However, with the increasing application and demand for USV racing, the aforementioned methods can no longer ensure that USVs complete tasks quickly and safely in environments with dynamic obstacles. This requires the trained policy to have strong performance to ensure that the USV can complete tasks at the fastest speed while also having robust capabilities to handle complex environments. Local navigation and RL control algorithms both have advantages in improving the performance of USV racing and obstacle avoidance. However, existing studies have only been tested with limited types of obstacles. This situation results in a lack of generalization of USV control strategies and leads to lower robustness when encountering unknown obstacles.

Thus, in this paper, we design a USV racing and obstacle avoidance method based on adversarial deep reinforcement learning (ADRL) (Tan et al., 2020). We also develop a simulation environment for this article's experiments with Unity3D (Juliani et al., 2020). The ray perception sensor is added to USV to get the obstacles' information in real-time. Inspired by Wang et al. (2024b), the state of ADRL is designed to make the USV travel safely at a high rate of speed. The reward function of ADRL is also designed to keep the reward of finishing the mission greater than hanging in tracks.

Unlike terrestrial or aerial domains where ADRL has been applied, the marine environment presents unique challenges that make ADRL particularly suited for USV applications. First, the fluid dynamics of water create continuous disturbances that require robust control strategies, making adversarial training beneficial for developing policies resilient to environmental uncertainties. Second, USV racing scenarios demand simultaneous optimization of speed and safety in confined waterways, where traditional DRL methods often fail to balance these competing objectives. Third, marine obstacles exhibit complex movement patterns influenced by currents and waves, necessitating adversarial training to expose the USV to diverse and unpredictable obstacle behaviors during training. These domain-specific characteristics distinguish our ADRL framework from previous applications in other robotic domains and existing DRL-based USV navigation approaches. The main contributions of this article are as follows.

1. We introduce an ADRL-based framework for USV autonomous racing and obstacle avoidance, where adversarial obstacle agents are explicitly incorporated during training. This enables the USV to learn robust and generalizable navigation strategies that outperform conventional DRL methods, especially in complex and dynamic environments.
2. To enhance the robustness of USV obstacle avoidance, ADRL is applied to train a policy for USV racing. Through adversarial training, the obstacle agent learns a diverse set of behaviors, which in turn improves the generalization of the USV's obstacle avoidance policy. We also design the states and reward function using information from the USV and its environment to ensure the USV races as quickly as possible.
3. We develop a simulation environment using Unity3D. The proposed approach is implemented and evaluated in this environment. Experimental results demonstrate improved USV racing speed and robustness in obstacle avoidance compared to baseline methods.

The remainder of this paper is organized as follows. Section 2 provides an overview of the background of ADRL, autonomous USV race and obstacle avoidance. Section 3 presents the proposed method for robust USV racing and obstacle avoidance using the ADRL algorithm. Section 4 describes the simulation environment, training procedures, and evaluation results. Section 5 discusses the limitations of the current approach and outlines promising directions for future research. Finally, Section 6 concludes the paper with a summary of contributions and findings.

2. Background

2.1. Adversarial deep reinforcement learning

ADRL integrates adversarial training techniques with DRL algorithms, aiming to develop agents capable of reliably performing tasks even when faced with adversarial attacks or environmental perturbations.

The motivation behind ADRL lies in the need to create control systems that are both reliable and secure, particularly for safety-critical applications such as autonomous vehicles. By training autonomous vehicles with ADRL, systems can be tested against adversarial driving scenarios, allowing for the identification of potential vulnerabilities. Additionally, ADRL is utilized to defend DRL agents against adversarial attacks. The primary challenges in ADRL are as below.

1. Developing effective adversarial attack strategies to assess the robustness of DRL agents.
2. Designing countermeasures and defense mechanisms to shield DRL agents from adversarial attacks.
3. Ensuring training stability and maintaining performance when incorporating adversarial components into the RL framework.

Current trends in ADRL involve the use of techniques such as cumulative prospect theory to model human risk perception in adversarial driving scenarios (Tan et al., 2020).

2.2. Autonomous USV racing

Autonomous USV racing involves the USV selecting the optimal route on different tracks to reach the target point in the shortest possible time. Autonomous USV racing has also emerged as a competitive sport, with events designed to test the speed, agility, and navigation accuracy of these vessels (Zissis et al., 2024). Races often introduce various obstacles, changing weather conditions, and require strategic decision-making. This field drives innovation and pushes the limits of what autonomous systems can achieve in marine technology and robotics.

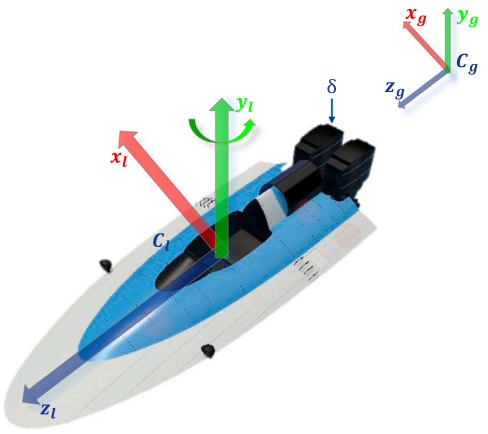


Fig. 1. Body-fixed frame and earth frame of USV.

Obstacle avoidance is a vital capability for autonomous USV racing to navigate safely in dynamic marine environments. USVs rely on a variety of sensors, including radar (Sun et al., 2024; Li et al., 2024b), LiDAR (Zhou et al., 2024; Villa et al., 2020) and camera (Cheng et al., 2023), to detect and classify obstacles in their surroundings (Chen and Wang, 2020). Integrating data from these multiple sensors enhances the reliability and accuracy of obstacle detection, allowing USVs to develop a comprehensive understanding of their environment. Onboard sensors also enable USVs to map the seafloor and identify potential hazards, such as submerged rocks or shipwrecks. Precise localization, often achieved through GPS and inertial measurement units (IMUs), is essential for determining the USV's position relative to obstacles.

Advanced algorithms, such as rapidly-exploring random trees (Karaman et al., 2011; Noreen et al., 2016) and A* search, empower USVs to plan collision-free paths and navigate around obstacles effectively (Li et al., 2024a). Real-time control systems enable USVs to react swiftly to dynamic obstacles, making necessary adjustments to their trajectories to avoid collisions. Machine learning techniques, including deep learning, are employed to classify obstacles based on sensor data, further improving the accuracy and dependability of obstacle detection (Wang et al., 2022). Systems based on RL allow USVs to learn from previous experiences, adapt to new situations and continuously optimize their obstacle avoidance strategies (Yan et al., 2021). Moreover, USV operations must comply with maritime regulations like the International Regulations for Preventing Collisions at Sea, which set the rules for vessel navigation (Fan et al., 2022; Ma et al., 2020). Developing and adhering to safety standards is essential for the safe integration of autonomous vessels into marine environments.

2.3. USV model

The dynamic modeling of USVs is a key point for understanding their behavior and developing effective control strategies. To describe and control USV conveniently, two frames are set as Fig. 1.

The frames' system follows the basic settings used in Unity3D, also called the left-handed system of frames. The frame C_l is the body-fixed frame of USV. z_l indicates the front direction of the vessel. x_l represents the right direction and y_l is the up direction. C_g is the earth frame. z_g, x_g and y_g represent the directions of the global environment. The position of USV in the global environment is set as the USV's center of mass. The angle is used to describe the USV's attitude in the earth frame. δ represents the rudder angle in the body-fixed frame.

The dynamics of USVs are influenced by various factors, including hydrodynamic forces, environmental conditions, and the vessel's physical characteristics. In the USV racing scenario, the position and heading angle of the USV are considered, while the external sea wind is not taken into account. The USV dynamic model (Xiao et al., 2018) is given by Eq. (1).

$$\begin{cases} \dot{z} = u\cos\psi - v\sin\psi \\ \dot{x} = u\sin\psi + v\cos\psi \\ \dot{\psi} = r \\ m_{11}\dot{u} - m_{22}vr + d_{11}u + d_{12}v + d_{13}r = \tau_u + X_{\mathrm{w}} \\ m_{22}\dot{v} + m_{11}ur + d_{22}v + d_{23}r = Y_{\mathrm{w}} \\ m_{33}\dot{r} + (m_{22} - m_{11})uv + d_{32}v + d_{33}r = \tau_r + N_{\mathrm{w}} \end{cases} \tag{1}$$

where (z, x, ψ) represents the USV's position and heading angle in the earth frame system, respectively. (u, v, r) represent the velocity and angular velocity of the USV in the body-fixed frame system. m_{11}, m_{22} and m_{33} represent the inertial mass parameters of the USV. d_{11}, d_{12}, d_{13}, d_{22}, d_{23}, d_{32} and d_{33} represent the hydrodynamic damping parameters of the USV. τ_u and τ_r represent the control inputs as the longitudinal thrust and torque, τ_r is proportional to δ. X_{w} denotes the lateral wind force acting on the USV, affecting its side-to-side movement. Y_{w} represents the longitudinal wind force, impacting the forward or backward motion of the USV. N_{w} indicates the yaw moment caused by the wind, influencing the vehicle's rotation or heading stability.

As shown above, many nonlinear dynamics cannot be precisely determined due to external unpredictable disturbances and uncertainties. Moreover, many parameters of the USV model require significant resources to obtain from the experiments.

3. Robust USV racing and obstacle avoidance with ADRL

In this section, an autonomous racing and obstacle avoidance policy for USV is developed using the ADRL algorithm. The section begins with an introduction to the Markov decision process (MDP) model for the USV racing scenario. Next, the MDP model for obstacle dynamics is outlined, followed by a detailed explanation of the obstacle avoidance strategy. Finally, a step-by-step guide is provided on applying the ADRL algorithm to train a USV racing policy with effective obstacle avoidance capabilities.

3.1. USV racing MDP model

The RL-based method is suitable in this situation because it does not require a precise model. Therefore, the USV dynamics can be described as an MDP with an unknown transition model, which is determined by selecting the appropriate states and actions. The MDP model for the racing USV with integrated obstacle avoidance is introduced, based on the robust ADRL framework. The core concept involves using the ADRL algorithm to iteratively train a policy that enables the USV to navigate a racecourse while avoiding obstacles, even under adversarial conditions where an obstacle agent actively challenges the USV control agent.

The interaction between the agent and the environment is typically modeled as an MDP, represented by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$. In this model, $\mathcal{S}$ denotes the set of states, $\mathcal{A}$ denotes the set of actions, $\mathcal{P}$ is the state transition probability function, $\mathcal{R}$ is the reward function, and γ is the discount factor. At each step, the agent gets the current state s_t from the environment, chooses an action a_t based on its policy π, obtains a reward r_t, and transitions to the next state s_{t+1} according to the state transition probability function $\mathcal{P}$. The learned policy aims to maximize the expected cumulative reward. $R_t = \Sigma_{t'=t}^{T_{\max}} \gamma^{t'-t} r_{t'}$ is the return. The state–action value function describes the expected return under policy π, which is defined as $Q^{\pi}(s,a) = \mathbb{E}_{\pi}[R_t | s_t, a_t]$.

The goal of the USV racing is to navigate a generated track safely and quickly while avoiding obstacles. To achieve this, the race USV MDP model represents the interaction between the USV, the adversarial obstacle, and the track environment. The state transition model in this MDP is governed by the previously mentioned USV control model. The state and action spaces are designed to make sure that the interaction between the agent and the environment can adhere to the Markov property. Thus, all states affecting the USV's motion during the race must be included. The following are defined based on the above principle.

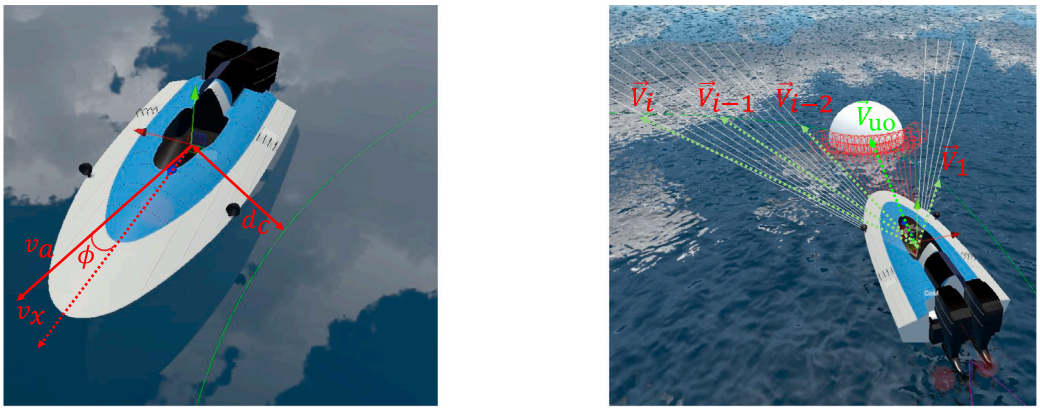


Fig. 2. State space of USV for racing MDP model. The green line in the left figure is the centerline of the track. v_x is the velocity of USV in the body-fixed frame system and v_a is the velocity of v_x along the tangent direction of the point on the USV closest to the centerline. ϕ is the angle between vectors v_x and v_a, representing the USV's heading relative to the green line. d_c is the shortest distance between the USV and the green line. $\vec{V}_{d_1}, \ldots, \vec{V}_{i-2}, \vec{V}_{i-1}, \vec{V}_i$ is the vector of USV's forward observations. $\vec{V}_{\text{uo}}$ is the direction vector from the USV to the obstacle.

3.1.1. State space of USV

The states of the USV are categorized into three groups. The first group consists of the USV's kinematic and control states. The longitudinal velocity v_x provides critical information about the USV's forward speed, essential for racing performance assessment and timely obstacle avoidance. The turning rate ω indicates the USV's rotational velocity, crucial for predicting trajectory changes and executing evasive maneuvers. The rudder angle δ represents the current control input status, allowing the agent to understand the relationship between control actions and resulting motion. Together, these states enable the agent to comprehensively understand the USV's dynamic state and control authority, forming the foundation for effective racing and obstacle avoidance decisions.

The second group comprises track-relative positioning data essential for precise navigation guidance. The track alignment angle ϕ measures the deviation between the USV's heading vector and the track's tangent direction, serving as a critical indicator for course correction decisions. When $\phi = 0$, the USV is perfectly aligned with the intended racing line, enabling maximum forward progress; conversely, when $|\phi| > \pi/2$, the system can immediately identify that the vessel is moving in the wrong direction, triggering appropriate course correction protocols. The track distance metric d_c quantifies the USV's lateral deviation from the centerline, providing essential information for maintaining the optimal racing line and avoiding track boundaries. Together, these variables enable the agent to continuously assess its racing trajectory quality and implement precise steering adjustments to optimize both speed and safety throughout the course.

To enable the USV to perceive the racing task, the remaining route length d_{le} is calculated and normalized using the total path length d_{to}. N_{FO} forward-observation vectors is another states group, and can capture the curvature of the upcoming section of the racetrack. As depicted in Fig. 2, these forward-observation vectors $\vec{V}_{d_i}$ are evenly distributed in front of the USV, enabling it to assess the track's curvature ahead. The vectors also indicate the USV's limited field of view, a situation referred to as racing under partial observation, which is formally defined as:

$$\vec{V}_{d_i} = \mathbf{R}_e^b [X_{d_i} - X, Y_{d_i} - Y]^T \tag{2}$$

where $\mathbf{R}_e^b$ represents the rotation matrix that transforms frames from the earth frame to the body-fixed frame. The point X_{d_i} and Y_{d_i} correspond to the ith forward-observation point in the earth frame, while X and Y denote the USV's position in the earth frame. Consequently, the vector of forward observations is expressed as $\mathbf{V}_{\text{FO}} = [\vec{V}_{d_1}, \vec{V}_{d_2}, \ldots, \vec{V}_{d_{N_{\text{FO}}}}]$.

The forward-observation vectors $\vec{V}_{d_i}$ are derived from the Unity3D ray perception sensor, where each ray corresponds to one observation point. With Ray Per Direction = 15 (Table 1), the sensor casts 15 rays within the Max Ray Degrees = 30° field of view, each with Ray Length = 40 m. When a ray intersects with track boundaries or obstacles, the intersection point (X_{d_i}, Y_{d_i}) is recorded and transformed to the body-fixed frame to generate $\vec{V}_{d_i} = (x_i, y_i)$. If no intersection occurs within the ray length, a default maximum distance value is used, ensuring consistent input dimensionality.

The third group of states consists of sensor data, which reflects the detected obstacles in front of the USV. This sensor data includes the distance to the obstacle d_{obs} and the direction vector from the USV to the obstacle $\vec{V}_{\text{uo}}$. The primary purpose of this sensor data is to facilitate obstacle avoidance.

It is worth noting that external environmental factors such as wind forces and water currents, are not included in the state space since these elements are not modeled in the simulation environment. Thus, the state vector for the USV racing MDP model is defined as follows:

$$s_t = [v_x, \omega, \delta, d_c, \phi, \mathbf{V}_{\text{FO}}, d_{\text{obs}}, \vec{V}_{\text{uo}}, d_{\text{le}}/d_{\text{to}}] \tag{3}$$

To establish the explicit connection between theoretical state definitions and actual neural network inputs, the state vector s_t is mapped to a concrete input tensor structure. With $N_{\text{FO}} = 15$ forward-observation vectors (corresponding to Ray Per Direction in Table 1), each $\vec{V}_{d_i}$ contributes 2 components (x_i, y_i) in the body-fixed frame, and $\vec{V}_{\text{uo}}$ provides 2 directional components $(x_{\text{uo}}, y_{\text{uo}})$. The complete input tensor $\mathbf{s}_t \in \mathbb{R}^{39}$ is structured as:

$$\mathbf{s}_t = [v_x, \omega, \delta, d_c, \phi, \underbrace{x_1, y_1, \ldots, x_{15}, y_{15}}_{30 \text{ components}}, d_{\text{obs}}, x_{\text{uo}}, y_{\text{uo}}, d_{\text{le}}/d_{\text{to}}]^T \tag{4}$$

where the forward-observation vectors are flattened from $\mathbf{V}_{\text{FO}} = [\vec{V}_{d_1}, \ldots, \vec{V}_{d_{15}}]$ to $[x_1, y_1, \ldots, x_{15}, y_{15}]$ for direct neural network processing. This 39-dimensional vector serves as the input to both the actor and critic networks in the TD3 algorithm, ensuring consistent state representation throughout the training process.

3.1.2. Action space of USV

The USV is controlled by two inputs: the rudder angle δ and the throttle $thr \in [-1, 1]$. For the racing scenario in this study, we adopt a single-input control strategy by setting the throttle to maximum power ($thr = 1$) and using only the normalized rudder angle as the action input. The rudder angle δ is normalized to u_δ to map the physical angle range to the standard RL action range. Therefore, the action space of the USV is defined as

$$a_t = [u_\delta] = [\delta/\pi] \in [-1, 1] \tag{5}$$

The action tensor $\mathbf{a}_t \in \mathbb{R}^1$ represents a single continuous value that directly controls the USV's steering behavior. The neural network's actor output is mapped to the physical rudder angle through the inverse transformation $\delta = u_\delta \times \pi$, where u_δ is clipped to the range $[-1, 1]$ during training. This ensures that the physical rudder angle remains within the feasible range $[-45°, +45°]$ as specified in Table 1, providing direct correspondence between the network output and actual control input.

3.1.3. Reward function of USV

The reward function of the USV is designed to promote both safe navigation and efficient racing performance. This function consists of five components. At each time step, the USV racing policy aims to maximize the vehicle's effective forward velocity, defined by the velocity reward $r_{\text{vel}} = v_a = v_x \cos\phi$, which represents the projection of the USV's longitudinal velocity along the track direction. This reward structure is specifically designed to mitigate the sparse reward problem in RL by providing continuous feedback through the velocity reward r_{vel} at every time step, rather than only offering rewards upon task completion. The continuous nature of the velocity reward creates a dense reward signal that guides the learning process even when the agent is far from completing the full racing task, enabling more efficient policy optimization and faster convergence to optimal behavior.

To ensure trajectory safety, we implement significant negative rewards: $r_{\text{out}} = -100$ for leaving the track area, $r_{\text{ww}} = -100$ for wrong-way navigation, and $r_{\text{obs}} = -100$ for obstacle collision. The magnitude of these negative rewards -100 substantially exceeds typical velocity rewards, which generally range from 0 to 10 in our racing scenarios. This deliberate imbalance creates a strong disincentive for unsafe behaviors, effectively prioritizing safety over speed in the learning process. When the USV operates safely within the track boundaries and avoids obstacles, these penalty terms are nullified $r_{\text{out}} = r_{\text{ww}} = r_{\text{obs}} = 0$.

The task of USV racing is to reach the end line of the track in the shortest time. To incentivize the USV to complete the racing task as quickly as possible, a positive reward is assigned for task completion. First, the base reward R_{base} represents the reward for completing the racing task, which needs to be large enough to ensure the USV is properly motivated to reach the finish line. The target completion time T_{target} is set and the actual completion time T_{actual} is used to calculate the reward. If the USV completes the task faster than the target time, it receives a higher reward. If the USV does not complete the task, the completion reward is set to 0. Therefore, the completion reward is defined as follows:

$$r_{\text{comp}} = \begin{cases} \frac{T_{\text{target}}}{T_{\text{actual}}} \times R_{\text{base}} & \text{if task completed} \\ 0 & \text{if task not completed} \end{cases} \tag{6}$$

The total reward function for the USV is a combination of the velocity reward, the negative rewards for unsafe behaviors and the positive reward for completion. The total reward function is defined as:

$$r_t = r_{\text{vel}} + r_{\text{out}} + r_{\text{ww}} + r_{\text{obs}} + r_{\text{comp}} \tag{7}$$

This comprehensive reward function ensures that the USV is encouraged to drive fast, stay on the track, avoid wrong-way driving, avoid obstacles and complete the race as quickly as possible.

3.2. Adversarial obstacle MDP model

In this subsection, the obstacle MDP model is introduced. The obstacle MDP model is defined by a tuple $\langle S_{\text{obs}}, \mathcal{A}_{\text{obs}}, \mathcal{P}_{\text{obs}}, \mathcal{R}_{\text{obs}}, \gamma_{\text{obs}} \rangle$ that is similarly to the USV MDP model.

Dynamic obstacle's motion is controlled by the adversarial agent, which attacks the USV agent by colliding with the USV agent. The goal of the obstacle MDP model is to learn a policy that minimizes the USV agent's reward, which means adversarial obstacle has a stronger disturbance to the USV agent. The progress of training also can make much better experiences so that the USV racing policy's robustness can be improved. The state, action, and reward function of the obstacle MDP model are defined as follows.

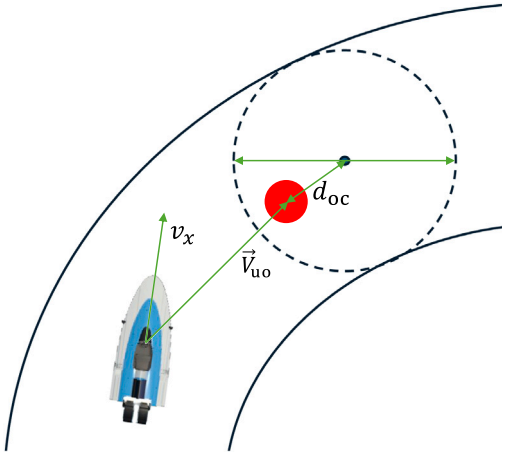


Fig. 3. State space of obstacle for adversarial MDP model. The red circle is the obstacle. The dashed circle represents the permissible movement area for the obstacle. v_x is the USV's longitudinal velocity. $\vec{V}_{\text{uo}}$ is the relative direction from the USV to the obstacle, with its magnitude representing the relative distance d_{uo}. d_{oc} is the distance between the obstacle and the center of its permitted movement area.

3.2.1. State space of obstacle

The obstacle state representation is structured into three distinct groups. The first group characterizes the obstacle's positional information. Since adversarial obstacles possess mobility capabilities, both their initial position and permissible movement area must be defined and constrained. In our implementation, the movement is restricted to a circular area with a radius equal to half the track width. To enforce this spatial constraint effectively, we define the parameter d_{oc} as the distance between the obstacle's current position and the center of its permitted circular movement area, as illustrated in Fig. 3.

The second group of states describes the relative information between the USV and the obstacle. The relative distance d_{uo} and the relative direction $\vec{V}_{\text{uo}}$ from the USV to the obstacle are used to indicate the relative position of the USV and the obstacle. The third group of states describes the USV's motion. The USV's longitudinal velocity v_x is used to indicate the USV's motion. Therefore, the state vector of the obstacle MDP model is defined as:

$$s_t^{\text{obs}} = [d_{\text{oc}}, d_{\text{uo}}, \vec{V}_{\text{uo}}, v_x] \tag{8}$$

Similar to the USV state representation, the obstacle state vector is mapped to a concrete input tensor for neural network processing. The direction vector $\vec{V}_{\text{uo}}$ is decomposed into its (x, y) components, resulting in the obstacle input tensor $\mathbf{s}_t^{\text{obs}} \in \mathbb{R}^5$:

$$\mathbf{s}_t^{\text{obs}} = [d_{\text{oc}}, d_{\text{uo}}, x_{\text{uo}}, y_{\text{uo}}, v_x]^T \tag{9}$$

This 5-dimensional vector provides the adversarial obstacle agent with sufficient information to make strategic movement decisions while maintaining computational efficiency during training.

3.2.2. Action space of obstacle

The adversarial obstacle requires movement capabilities to effectively challenge the USV agent. To simplify the obstacle's behavior while maintaining sufficient degrees of freedom, the obstacle's action is

defined as a normalized movement offset vector O_m in two dimensions. This allows the obstacle to move in any direction within its permitted area, creating diverse and unpredictable challenges for the USV. The movement offset is applied to the obstacle's position at each time step, allowing for continuous adjustment of the obstacle's trajectory to intercept the USV. Therefore, the action space of the obstacle is defined as

$$a_t^{\text{obs}} = [O_m] = [o_x, o_y] \in [-1, 1] \tag{10}$$

3.2.3. Reward function of obstacle

The reward function of the obstacle is designed to minimize the USV agent's reward. And the obstacle's reward shaping is $r_{\text{step}} = 0.1$ in every time step. This step reward, similar to the USV agent's step reward, aims to alleviate sparse rewards and encourage continuous movement during training. To encourage the obstacle to attack the USV agent, the collision reward is set to $r_{\text{col}} = 100$. This significant positive value incentivizes the obstacle to actively seek collision with the USV, creating challenging scenarios for the USV agent to learn robust avoidance strategies. $r_{\text{out}} = -100$ is set in case of the obstacle's moving out of the circle area, which prevents the obstacle from simply escaping the training environment and ensures meaningful interactions between the obstacle and USV agent. Therefore, the reward function is defined as:

$$r_{\text{obs}} = r_{\text{step}} + r_{\text{col}} + r_{\text{out}} \tag{11}$$

3.3. Training and implementation

3.3.1. Training procedures

The ADRL framework differs fundamentally from standard DRL approaches and prior RL-based obstacle avoidance methods in several critical aspects. Unlike conventional DRL methods that train a single agent to navigate in environments with static or predictably moving obstacles, our ADRL approach employs a dual-agent adversarial training paradigm where the obstacle actively learns to challenge the USV agent. This contrasts sharply with prior RL obstacle-avoidance methods, which typically treat obstacles as passive environmental components with predetermined or rule-based behaviors.

In standard DRL frameworks for USV navigation, the environment remains relatively static during training, with obstacles following fixed patterns or simple reactive behaviors. This approach often leads to policies that perform well in training scenarios but lack robustness when encountering novel obstacle behaviors in real-world deployment. Conversely, our ADRL methodology creates a continuously evolving adversarial environment where the obstacle agent actively adapts its strategies to exploit weaknesses in the USV's policy, forcing the development of more robust and generalizable navigation strategies.

The key methodological distinction lies in the training dynamics: while conventional approaches optimize a single objective function for obstacle avoidance, ADRL employs a minimax optimization framework where the USV agent maximizes its navigation performance while the adversarial obstacle agent minimizes it. This adversarial co-evolution process ensures that the USV encounters a diverse spectrum of challenging scenarios that would be difficult to manually design or predict, leading to superior generalization capabilities compared to methods trained against static or scripted obstacles.

Furthermore, unlike prior RL methods that require extensive domain knowledge to design appropriate obstacle behaviors for training diversity, our ADRL framework automatically generates challenging scenarios through the adversarial learning process, reducing the need for manual scenario engineering while ensuring comprehensive coverage of potential obstacle behaviors.

The training procedures in Algorithm 1 detail the process of updating the networks for both the USV and obstacle agents using the ADRL algorithm. Both the main USV agent and the adversarial obstacle agent are trained from scratch without any pre-training, learning simultaneously through their interactions.

The adversarial training protocol follows an alternating multi-agent learning framework where both agents maintain separate replay buffers, neural networks, and learning parameters. Specifically, the obstacle agent uses identical network architectures (actor–critic) and hyperparameters as the USV agent, but with a fundamentally different reward structure designed to maximize collisions rather than avoid them. The obstacle agent's learning rate, exploration noise, and network update frequencies are set identical to the USV agent (as shown in Table 3), ensuring balanced learning dynamics between agents. The key distinction lies in the reward formulation: while the USV receives negative rewards for collisions, the obstacle agent receives positive collision rewards, creating opposing optimization objectives that drive diverse adversarial behavior emergence through the alternating training process.

In the algorithm, both agents are initialized with their own actor–critic networks. These networks are trained in an alternating fashion, with each agent being trained for a specified number of steps defined by the T_{train} parameter. This alternating training approach is crucial for achieving balance between the agents. If the adversarial obstacle agent learns too frequently, it creates a constantly changing environment that forces the main USV agent to continually adapt rather than learn robust policies. Conversely, if the main USV agent learns too frequently, the adversarial obstacle agent cannot develop effective challenging strategies, diminishing training effectiveness. Therefore, selecting an appropriate alternating training step is essential as it ensures that during one agent's training period, the other agent's policy remains stable, maintaining relative environmental consistency. This balanced alternating approach facilitates the successful development of robust ADRL policies.

For the USV agent (main agent), the training process involves learning to navigate through the racecourse while avoiding obstacles. The USV's actor network selects actions based on the current state, while its critic network evaluates the quality of these actions.

By interacting with an adversarial obstacle that actively tries to collide with it, the USV is exposed to a wide variety of challenging scenarios, forcing it to learn more robust avoidance strategies than it would if trained with static or moving obstacles.

The obstacle agent (adversarial agent) is trained to maximize its reward by attempting to collide with the USV. Its actor network selects movements that are likely to intercept the USV's path, while its critic network evaluates the effectiveness of these interception attempts. The obstacle's behavioral diversity emerges through several key mechanisms: (1) action noise injection during training introduces stochastic variations in obstacle movements after policy sampling, allowing discovery of novel attack patterns; (2) experience replay ensures the obstacle learns from a diverse set of historical USV-obstacle interactions, preventing overfitting to current USV strategies; (3) the alternating training schedule forces the obstacle to continuously adapt its strategies as the USV policy evolves, leading to progressive strategy refinement that naturally generates behavioral complexity; and (4) the spatial constraints of the circular movement area require the obstacle to develop context-aware strategies depending on track geometry and USV approach patterns. As the USV becomes more adept at avoidance, the obstacle agent must develop increasingly sophisticated interception strategies, creating a self-improving cycle that enhances both agents' capabilities.

The algorithm ensures that the USV agent learns to navigate the track efficiently while avoiding obstacles, and the obstacle agent learns to generate adversarial examples to challenge the USV agent.

Algorithm 1 USV Racing and Obstacle Avoidance Training with ADRL

function UpdateNetworks(*networks*)
  Calculate target values
  Update critic network
  if $t \mod d = 0$ **then**
    Update actor networks
    Update critic and actor target networks
Initialize:
Initialize the environment
Randomly initialize parameters of actor-critic original and target networks
Initialize replay buffer D, agent step counter $t_{\text{agent}} = 0$
Set max episode number M, max timesteps $T_{\max}$, delay parameter d, alternating training steps T_{train}, target lap time T_{target} and base reward R_{base}
for $episode = 1, \ldots, M$ **do**
  Reset environment and get states s, s_{obs}
  for $t = 1, \ldots, T_{\max}$ **do**
    Select actions a_t and a_t^{obs} from policies π and π_{obs}
    Add exploration noise to a_t and a_t^{obs}
    Execute actions a_t and a_t^{obs}
    Observe reward r and next state s'
    Store transition (s, a, r, s') in D
    if $t_{\text{agent}} < T_{\text{train}}$ **then**
      Execute UpdateNetworks($USV\ networks$) ▷ *Update USV agent*
    else
      Execute UpdateNetworks($Obstacle\ networks$) ▷ *Update obstacle agent*
    t_{agent} += 1
    if $t_{\text{agent}} == 2 \times T_{\text{train}}$ **then**
      $t_{\text{agent}} = 0$

3.3.2. Implementation procedures

Algorithm 2 outlines the procedure for testing the policies of the USV and obstacle agents after training. This algorithm details the procedures to initialize the environment, load the trained policies, and execute the actions of both the USV and obstacle agents within the simulation environment. The implementation ensures that the USV agent can effectively reach the endline of the track while avoiding obstacles, thereby demonstrating the robustness and generalization of the trained policy.

4. Simulation and results

In this section, the simulation environment is built and the training procedures are described. To evaluate the performance of the ADRL algorithm for robust USV racing and obstacle avoidance, this study developed a simulation software with Unity3D and ML-Agents framework (Juliani et al., 2020). In the following, we first introduce the environment for simulation. Then, the training procedures of the ADRL algorithm are described. Finally, the evaluation results are presented and discussed about the performance of the ADRL algorithm.

4.1. Simulation environment

Unity3D has realistic physics and graphics rendering capabilities, making it an ideal platform for simulating USV racing scenarios. The ML-Agents framework provides a flexible and scalable environment for training intelligent agents using RL algorithms. The USV racing environment is designed to simulate a dynamic racecourse with obstacles, where the USV agent must navigate to reach the finish line while avoiding collisions with obstacles. To mitigate potential overfitting issues, incorporating diverse obstacle configurations and track layouts during training enhances the model's generalization capabilities. This diversity in training scenarios contributes to ADRL's superior performance across various test conditions. The environment includes the following components:

Algorithm 2 USV Racing and Obstacle Avoidance Implementation

Initialize:
Set the types of the track and the obstacle
Create USV environment progress
Load the trained USV policy π and obstacle policy π_{obs}
Set the max timesteps $T_{\max}$
Initialize the specified types' track and obstacle
Reset the states s for USV and s_{obs} for obstacle
for $t = 1, \ldots, T_{\max}$ **do**
  Select action a_t from USV policy π and a_t^{obs} from obstacle policy π_{obs}
  Execute actions a_t and a_t^{obs}
  if USV reaches target position **then**
    break

Table 1
USV physical and ray perception sensor parameters.

USV parameter	Value	Sensor parameter	Value
Mass	950 [kg]	Ray per direction	15
Max speed	35 [m/s]	Max ray degrees	30 [degree]
Rudder max angle	45 [degree]	Ray length	40 [m]
Rotation velocity	20 [degree/s]	Sphere cast radius	0.5 [m]

1. USV Agent: The initial position of the USV agent is always set at the beginning line of tracks. The power engine is integrated with rudders together. In this environment, the USV agent aims to complete the racecourse in the shortest time possible while avoiding obstacles.
2. Obstacle Agent: Obstacle can move in the pointed area. In this area, the obstacle agent will take adversarial move steps to hinder the USV agent's progress.
3. Obstacles: Static or dynamic obstacles are placed along the tracks to challenge the USV agent. Static obstacles remain stationary, while dynamic obstacles move within a predefined area. The movable obstacles are either controlled by a policy with fixed moving speed or by an adversarial policy. Obstacles controlled by fixed-speed policies can create additional challenges.
4. Tracks: The tracks' features varying layouts, including straight sections, curves, and obstacles as described above. The types of tracks and obstacles must be specified precisely for training and implementation. Figs. 4 to 7 show all types of tracks and obstacles, the red spheres are obstacles. In the track figures, the random tracks are typically divided into two classes: one where the target point is on the right of the start point, and the other where it is on the left. Therefore, two images are selected to display the random track type. The total distance of the random track is around 110 m.

The ray perception sensor in Unity3D is used to detect obstacles and track boundaries. The sensor parameters are adjusted to optimize the USV agent's perception of the environment, enabling it to make informed decisions during racing. The sensor parameters include the number of rays per direction, the maximum ray degrees, the ray length, and the sphere cast radius. The physical and sensor parameters of the USV are provided in Table 1.

Table 2 summarizes the mapping between mathematical state/action definitions and neural network tensor dimensions in ADRL. To improve readability, the table is split by agent and type.

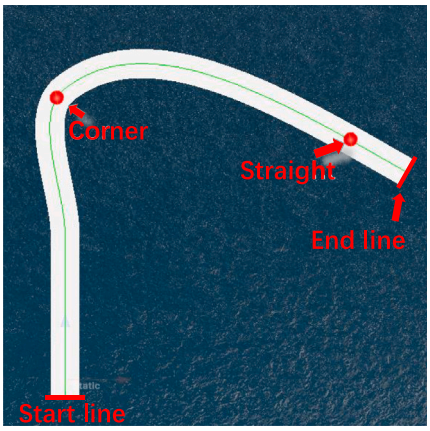


Fig. 4. Random path examples (a).

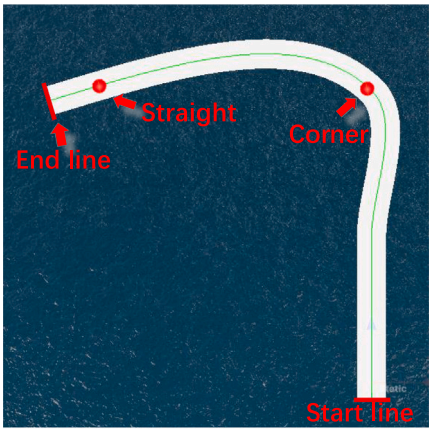


Fig. 5. Random path examples (b).

Table 2
State and action tensor dimensions for neural network implementation.

Type	Dim.	Components
USV agent		
State	$\mathbb{R}^{39}$	v_x, ω, δ, d_c, ϕ, x_1–x_{15}, y_1–y_{15}, d_{obs}, x_{uo}, y_{uo}, $d_{\text{le}}/d_{\text{to}}$
Action	$\mathbb{R}^1$	$u_\delta = \delta/\pi$
Obstacle agent		
State	$\mathbb{R}^5$	d_{oc}, d_{uo}, x_{uo}, y_{uo}, v_x
Action	$\mathbb{R}^2$	o_x, o_y

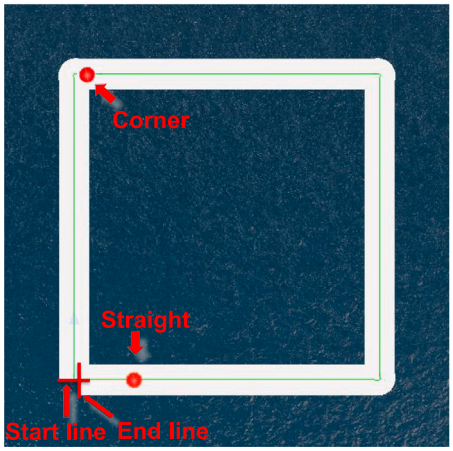


Fig. 6. Square shape path. The total distance of the track is 200 m.

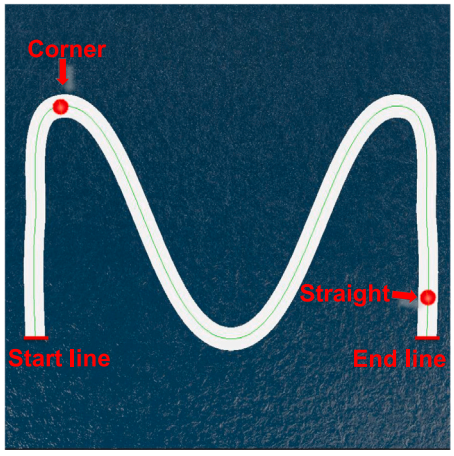


Fig. 7. W-shape path. The total distance of the track is 247 m.

Table 3
Training parameters with TD3 algorithm and Unity3D environment.

Parameter	Value	Parameter	Value
Quality level	0	R_{base}	2000
T_{target} for square path	20	T_{target} for W-shape path	40
Alternating training steps	10	Learning rate τ	0.001
Discount factor γ	0.99	Replay buffer size D	8×10^6
Batch size	512	Policy noise	0.2
Noise clipping	0.5	Target net update rate	2

4.2. Training and implementation with TD3

The proposed ADRL algorithm is based on the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm (Fujimoto et al., 2018), which is a state-of-the-art DRL algorithm for continuous control tasks. The TD3 algorithm addresses several key challenges in DRL, such as overestimation bias and sample inefficiency, making it well-suited for training the USV agent to navigate the racecourse and avoid obstacles effectively.

By leveraging the TD3 algorithm, our ADRL approach aims to enhance the performance and reliability of autonomous USV racing and obstacle avoidance, providing a robust solution for dynamic and adversarial environments.

The training process consists of the procedures of Algorithm 1. As for TD3 training procedures, the USV agent are trained in the same environment as ADRL, but the obstacle is not an agent but a static one. The parameters of the TD3 algorithm, which are important for the training process, are listed in Table 3.

The implementation procedures for USV racing and obstacle avoidance are detailed in Algorithm 2. The USV agents with ADRL and TD3 policies are tested in the same environment. The obstacle is non-agent type and moves in a series of directions.

To validate the robustness and generalization of the USV, various types of obstacles and tracks are designed, as described in Section 4.1. The trained policies for the USV and obstacle are loaded as controllers. The USV agent and obstacle agent are then placed in the environment with specified path types and obstacle configurations. To ensure the comparability of the results, the obstacle type in the test is set to move in a series of directions. It is important to note that during training, ADRL uses intelligent adversarial obstacles that actively learn to challenge the USV, while DRL trains against static obstacles. However, during testing, both ADRL and DRL policies are evaluated against the same standardized obstacle movement patterns (predefined directional sequences), ensuring fair performance comparison and highlighting the generalization benefits of adversarial training.

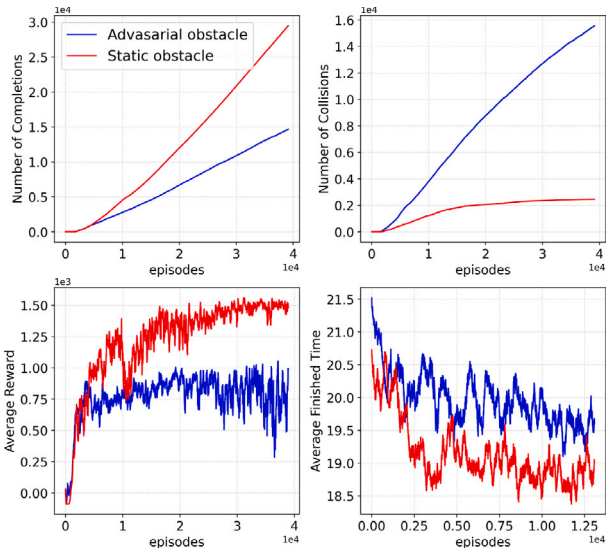


Fig. 8. Training results of USV on random paths with obstacles in the straight sections of the path.

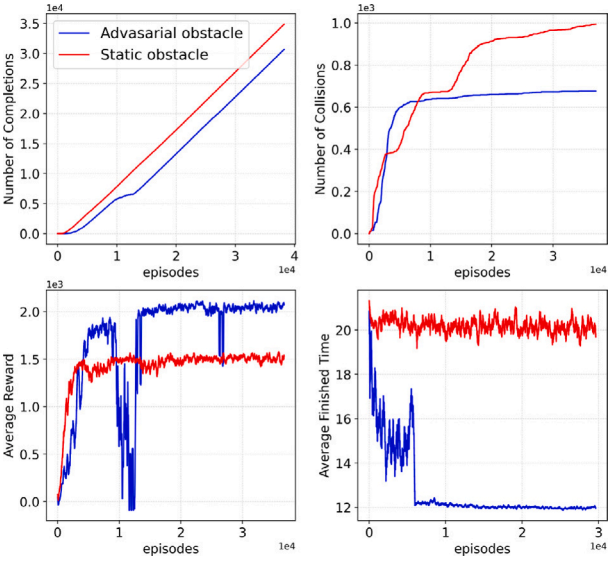


Fig. 9. Training results of USV on random paths with obstacles in the curved sections of the path.

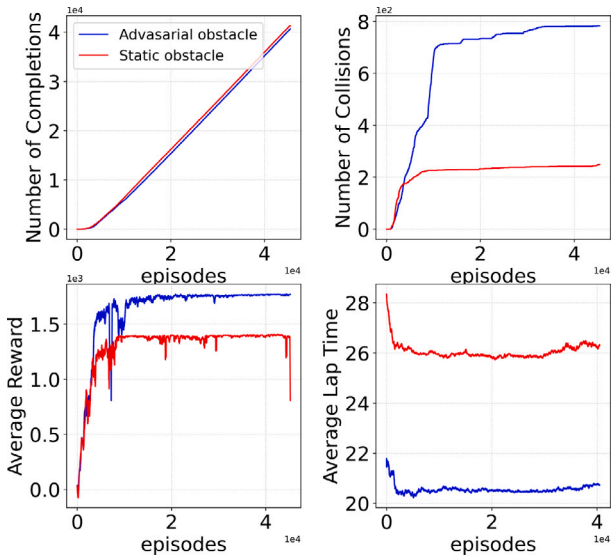


Fig. 10. Training results of USV on square paths with obstacles in the straight sections of the path.

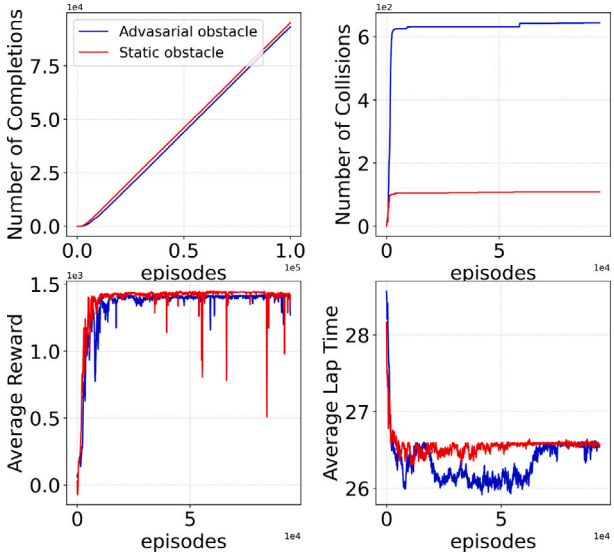


Fig. 11. Training results of USV on square paths with obstacles in the curved sections of the path.

4.3. Evaluation results

The performance of the USV agent is evaluated based on its ability to navigate the track efficiently while avoiding collisions with obstacles. The evaluation process involves multiple test scenarios, each featuring different track layouts and obstacle configurations. Key metrics used to assess performance include the number of completions, the number of collisions, and the average time to finish task.

4.3.1. Evaluation training results

The training results illustrated in Figs. 8 to 13 provide a comprehensive analysis of the performance comparison between ADRL and traditional DRL approaches. These figures present key metrics including completion rates, collision frequencies, average reward curves, and task completion times across various track configurations and obstacle placements.

During training, ADRL typically experiences more collisions and fewer completions than traditional DRL, reflecting the increased environmental complexity introduced by adversarial obstacles. On random paths, ADRL initially shows higher collision counts, but these gradually decrease as training progresses, demonstrating the algorithm's ability to develop increasingly effective avoidance strategies when facing dynamic adversarial obstacles.

The average reward curves confirm that ADRL initially yields lower rewards than DRL due to the challenging nature of its training environment. This is expected, as the agent must navigate an environment where obstacles actively attempt to hinder its progress. During early training phases, ADRL's reward and completion curves often exhibit significant oscillations as the agent balances exploration and exploitation while adapting to adversarial challenges.

ADRL's convergence speed is typically slower than traditional DRL because adversarial obstacles substantially increase environmental complexity, requiring more training iterations for the agent to learn

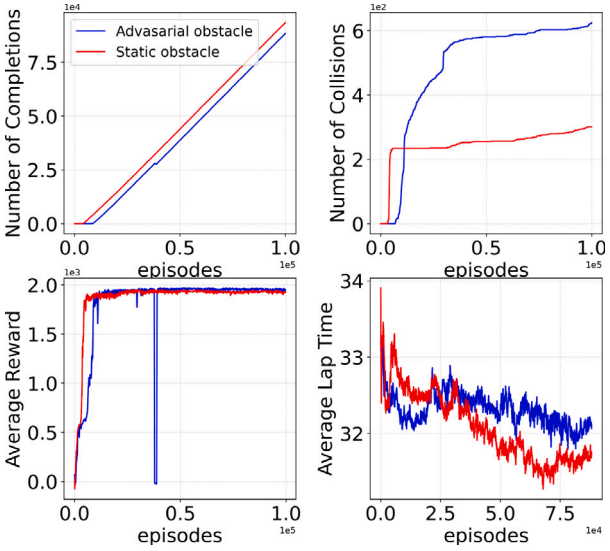


Fig. 12. Training results of USV on W-shape paths with obstacles in the straight sections of the path.

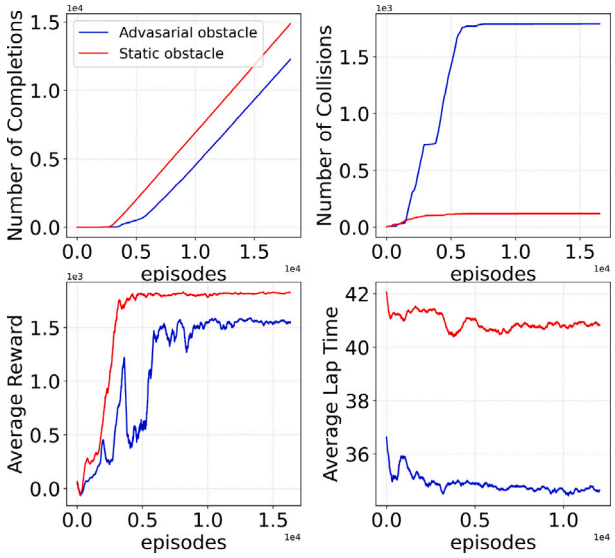


Fig. 13. Training results of USV on W-shape paths with obstacles in the curved sections of the path.

effective strategies. However, this additional training complexity ultimately produces more robust policies that outperform traditional DRL in complex environments, as demonstrated in subsequent implementation tests.

Despite facing more challenging conditions during training, the average completion time curves show that ADRL achieves competitive completion times, particularly on complex tracks with dynamic obstacles. This indicates that while ADRL accepts lower completion rates and rewards during training, it develops policies that maintain efficient navigation speed in complex environments.

A critical consideration for real-world deployment is the safety implications of training-phase collisions observed in Figs. 8 to 13. While ADRL experiences more collisions during training compared to traditional DRL, this apparent disadvantage actually contributes to enhanced safety in deployment. The training collisions serve as essential learning experiences that expose the USV agent to failure modes and recovery strategies that would be difficult to encounter in static-obstacle training environments.

The adversarial training paradigm deliberately creates challenging scenarios where collisions are inevitable during early learning phases, forcing the agent to develop more sophisticated avoidance behaviors. This controlled exposure to failure during training translates to superior collision avoidance capabilities during deployment, as evidenced by the dramatic reduction in test-phase collisions shown in Table 4.

Furthermore, the training collision patterns demonstrate convergence toward safer behavior over time, indicating that the learned policies internalize safety-critical decision-making rather than merely memorizing specific collision-avoidance maneuvers. The oscillatory nature of collision curves during training reflects the dynamic adaptation between USV and obstacle agents, ensuring the final policy is robust to diverse adversarial behaviors rather than optimized for a single static scenario. This comprehensive exposure to challenging conditions during training significantly reduces the likelihood of unexpected collisions in real-world deployment, where novel obstacle behaviors may emerge.

4.3.2. Evaluation performance metrics

The random function used in the Unity environment allows for the setting of a seed value, ensuring that the random number sequences generated with the same seed are consistent. This feature enables reliable comparisons and analyses of test results conducted under random paths. By utilizing a fixed seed of the random function, the environment maintains deterministic behavior, which means every run with that seed will produce the same sequence of random numbers. This consistency is particularly advantageous when evaluating the performance of different algorithms in identical conditions, as it isolates the effects of the algorithms themselves from the variability introduced by randomness. Consequently, any differences in the results of experiments can be attributed to the algorithms' effectiveness rather than randomness in the environment.

To rigorously evaluate the USV agent's performance in autonomous racing and obstacle avoidance scenarios, we conducted comprehensive testing across multiple environments with varying complexity levels: random paths, square-shaped paths, and W-shaped paths, each featuring obstacles in both straight sections and curved sections. Three distinct control methodologies were implemented for comparative analysis: our proposed ADRL approach, a conventional DRL framework, and a classical control strategy combining Line of Sight (LOS) guidance (Wu et al., 2025) with PID control and Artificial Potential Field (APF) (Pan et al., 2019) as baseline. Due to the extensive variety within the random path category, the LOS+PID+APF (LPA) method was applied only to the square-shaped paths and W-shaped paths for a more focused comparison.

More critically, LPA methods fundamentally require global path information for effective operation: LOS guidance needs complete target waypoint sequences, APF requires full environmental maps for potential field computation, and PID control demands precise reference trajectories. In contrast, both DRL and ADRL operate under partial observability constraints, relying only on local sensor observations to make navigation decisions. Random paths, with their infinite geometric variations and unpredictable curvature patterns, would require individual manual tuning of LPA parameters for each specific track configuration, making systematic comparison impractical. The structured nature of square and W-shaped paths allows for consistent LPA parameter configuration, enabling meaningful performance comparison under equivalent operational constraints.

To address the challenge that USVs employing LPA cannot safely navigate curves at high velocities, we implemented an adaptive throttle modulation system. This mechanism continuously evaluates the product of instantaneous velocity and imminent path curvature; when this value exceeds a predetermined safety threshold, the throttle is automatically attenuated to ensure secure navigation, otherwise maintaining maximum propulsion for optimal racing performance. The evaluation

Table 4
Performance metrics comparison between DRL, ADRL, and LPA for different route types and obstacle positions (5 seeds, 1000 episodes every seed; all methods tested with identical obstacle movement patterns - series of predefined directions for fair comparison).

Obstacle type	Completions	Collisions	Avg. Time
Straight sections			
LPA			
Square-shape	433.20 ± 6.88	494.60 ± 10.54	44.68 ± 0.02
W-shape	677.80 ± 3.97	322.20 ± 3.97	56.53 ± 0.02
DRL			
Random-shape	706.40 ± 26.70	172.20 ± 8.86	**17.02 ± 0.29**
Square-shape	456.40 ± 86.19	**174.20 ± 54.90**	**23.56 ± 1.28**
W-shape	727.60 ± 95.17	198.80 ± 11.20	38.46 ± 5.33
ADRL			
Random-shape	**829.40 ± 25.54**	**46.40 ± 43.04**	17.14 ± 0.89
Square-shape	**787.40 ± 55.25**	182.80 ± 57.47	24.11 ± 0.17
W-shape	**839.40 ± 75.80**	**121.60 ± 24.78**	**37.73 ± 5.60**
Curved sections			
LPA			
Square-shape	437.40 ± 5.04	229.60 ± 17.92	43.44 ± 0.04
W-shape	289.20 ± 63.86	643.60 ± 195.56	54.90 ± 0.20
DRL			
Random-shape	671.20 ± 22.89	237.40 ± 17.81	17.03 ± 0.69
Square-shape	811.80 ± 213.93	60.80 ± 10.93	21.78 ± 2.41
W-shape	506.00 ± 131.30	269.00 ± 17.18	38.83 ± 5.94
ADRL			
Random-shape	**889.60 ± 28.11**	**3.80 ± 2.32**	**15.74 ± 1.17**
Square-shape	**945.80 ± 81.44**	**1.60 ± 3.20**	**21.78 ± 2.02**
W-shape	**879.20 ± 14.92**	**0.40 ± 0.49**	**31.38 ± 2.51**

framework employed three critical performance metrics: completion rate (successful task executions), collision frequency (obstacle strikes), and average task completion time (temporal efficiency).

The test performance is presented in Table 4, where **bold** values highlight superior performance metrics. Each configuration was tested with 5 different seeds. The reported values represent means and standard deviations calculated across these five seeds, confirming that the performance metrics are statistically significant rather than random variations. This methodical approach ensures reliability and reproducibility of our findings, demonstrating that ADRL's superior performance remains consistent across multiple experimental trials. The comprehensive comparison between DRL, ADRL, and LPA clearly demonstrates ADRL's substantial advantages in autonomous navigation across diverse challenging scenarios. ADRL consistently exhibits superior performance, particularly in completion rates and collision avoidance, establishing it as a more robust and adaptable solution for autonomous USV racing in complex maritime environments.

One of the most striking differences between the algorithms is in their ability to avoid collisions, especially in curved routes. For example, on a random-shape path with obstacles in curved sections, ADRL registered only 3.80 ± 2.32 collisions, while DRL recorded 237.40 ± 17.81 collisions–a reduction of over 98%. The low standard deviation in ADRL's collision metric 2.32 compared to DRL's higher value 17.81 indicates not only superior performance but also greater consistency and reliability across test episodes. This pattern continues for square-shape paths with curved obstacles, where ADRL recorded just 1.60 ± 3.20 collisions compared to DRL's 60.80 ± 10.93, and for W-shape paths with curved obstacles, where ADRL achieved near-perfect performance with only 0.40 ± 0.49 collisions versus DRL's 269.00 ± 17.18 and LPA's 643.60 ± 195.56. The remarkably small standard deviation of 0.49 in the W-shape curved scenario demonstrates ADRL's exceptional consistency in challenging environments, while LPA's substantial standard deviation of 195.56 reveals significant performance instability.

ADRL also significantly outperforms both DRL and LPA in completion rates across all test scenarios. For random-shape paths with obstacles in straight sections, ADRL completed 829.40 ± 25.54 episodes successfully, compared to DRL's 706.40 ± 26.70 episodes–a 17.4% improvement with comparable standard deviations, indicating reliable performance gains. The performance gap widens for square-shape paths with straight obstacles, where ADRL achieved 787.40 ± 55.25 completions versus DRL's 456.40 ± 86.19 and LPA's 433.20 ± 6.88, representing increases of 72.5% and 81.8% respectively. Notably, ADRL's standard deviation 55.25 is substantially lower than DRL's 86.19, demonstrating more consistent performance despite achieving significantly higher completion rates. While LPA shows the lowest standard deviation 6.88, this comes at the cost of much poorer overall performance. This trend is consistent across all test environments, with ADRL demonstrating particularly strong performance in curved routes where it achieved 945.80 ± 81.44 completions on square-shape paths and 879.20 ± 14.92 completions on W-shape paths.

Regarding time efficiency, the algorithms show competitive performance across different scenarios. In straight routes, DRL sometimes achieves marginally faster completion times, such as 17.02 ± 0.29 s versus ADRL's 17.14 ± 0.89 s on random-shape paths, and 23.56 ± 1.28 s versus ADRL's 24.11 ± 0.17 s on square-shape paths. The slightly higher standard deviation in ADRL's completion time for random-shape paths (0.89 s compared to DRL's 0.29 s) suggests more variability in its navigation strategies, likely due to its prioritization of safety over speed. However, in square-shape paths, ADRL demonstrates remarkably consistent timing (0.17 s standard deviation versus DRL's 1.28 s), indicating more stable performance in structured environments.

ADRL demonstrates superior efficiency in more complex scenarios, particularly in curved routes where it achieves 15.74 ± 1.17 s, 21.78 ± 2.02 s, and 31.38 ± 2.51 s on random-shape, square-shape, and W-shape paths respectively–outperforming both DRL and LPA. The standard deviations for ADRL in these challenging scenarios (1.17 s, 2.02 s, and 2.51 s) are notably lower than or comparable to those of DRL (0.69 s, 2.41 s, and 5.94 s), indicating that ADRL maintains consistent performance even in complex environments. The traditional LPA method consistently shows significantly longer completion times, ranging from 43.44 ± 0.04 s to 56.53 ± 0.02 s across different tracks, with extremely low standard deviations that reflect its deterministic nature but inflexible approach.

These results indicate that ADRL not only navigates more safely with dramatically fewer collisions but also maintains competitive or superior time efficiency while achieving significantly higher completion rates. The performance advantages of ADRL are particularly pronounced in more challenging environments with curved sections and complex obstacle arrangements. The consistent performance across different track configurations, as evidenced by the generally lower standard deviations in critical metrics, demonstrates ADRL's strong generalization capabilities, making it a reliable solution for real-world USV applications where environmental conditions may vary unpredictably.

To provide a more intuitive analysis of ADRL performance, the USV trajectory curves for square and W-shaped routes with static obstacles are presented in Figs. 14 and 15. The total distance of the square track is 500 m, and w-shape one is 618 m. In the square path, ADRL completes the trajectory in 83.6 s, whereas DRL completes it in 86.3 s. In the w-shape path, ADRL completes the trajectory in 90.3 s, whereas DRL completes it in 93.1 s. ADRL consistently completes tasks in less time than DRL on various paths, indicating greater efficiency in path planning and obstacle avoidance. ADRL better handles static obstacles in complex paths, reducing collision risk and enhancing navigation safety. The trajectories generated by ADRL are smoother, avoiding unnecessary detours and optimizing path selection.

The superior performance of ADRL across multiple metrics–collision avoidance, task completion and time efficiency–demonstrates its robustness and adaptability to complex, dynamic environments. While DRL is capable of handling simpler scenarios to some extent,

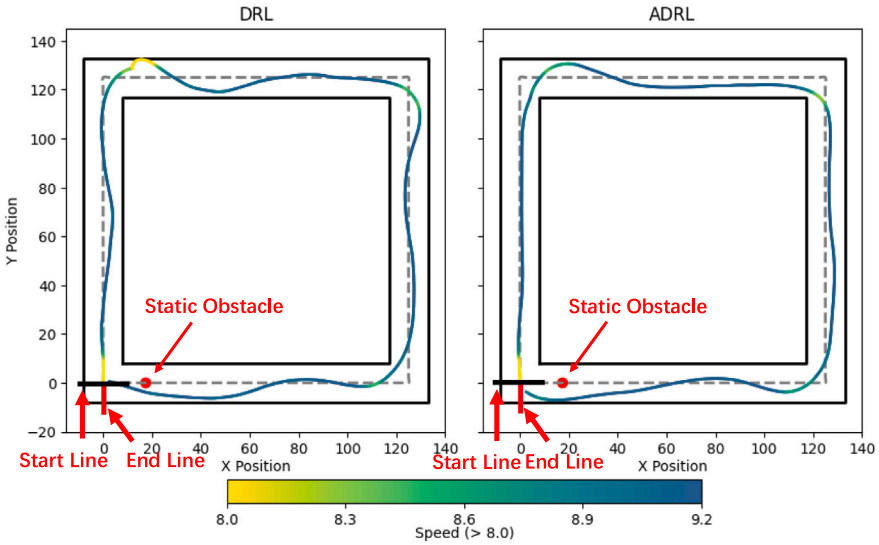


Fig. 14. Trajectories of USV on square paths with the static obstacle in the straight section of the path. (**Lap time: ADRL-83.6 s, DRL-86.3 s. Speeds above 8.0 m/s are shown in the colorbar legend, while speeds below 8.0 m/s are shown in yellow.**)

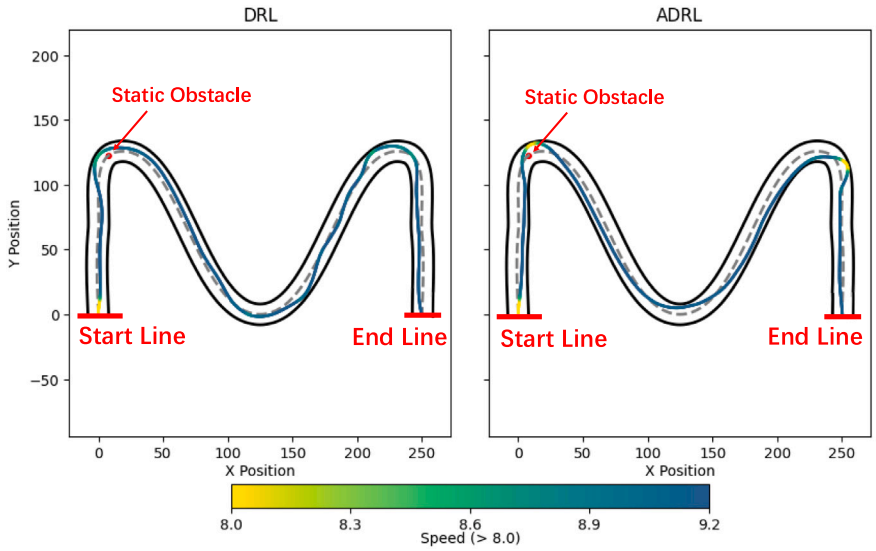


Fig. 15. Trajectories of USV on W-shape path with the static obstacle in the curved section of the path. (**Lap time: ADRL-90.3 s, DRL-93.1 s. Same color scheme as in** Fig. 14)

its performance drops significantly in more intricate paths with sharp turns or densely packed obstacles. ADRL, on the other hand, maintains high levels of performance regardless of the complexity of the environment. This makes ADRL particularly suited for real-world applications where autonomous systems must navigate unpredictable and constantly changing environments, such as marine environments with moving obstacles.

In summary, ADRL significantly enhances the capabilities of autonomous USVs, providing a more efficient and reliable solution than traditional DRL approaches. Its superior ability to avoid collisions, complete tasks efficiently and navigate complex environments in less time.

4.3.3. Evaluation parameter sensitivity

To investigate the parameter sensitivity of the ADRL algorithm, we conducted a comprehensive analysis focusing on the alternating training steps between the main agent and the adversarial agent, testing three different values: 1, 10, and 20. Unlike conventional RL parameters such as discount factor γ, learning rates, or network architectures, the alternating training step parameter is unique to our adversarial approach and significantly impacts performance. Standard RL parameters have been extensively studied in prior research with well-established conclusions. The USV agent's performance was evaluated across multiple metrics, including completion rate, collision frequency, cumulative reward, and average lap time. The results are illustrated in Figs. 16 through 19.

In the Average Lap Time curves, some curves remain at zero throughout the training process, while others appear only partially. The zero-valued curves indicate training configurations where the USV agent failed to complete any tasks during those episodes, resulting in no valid lap time data. Similarly, partially visible curves represent cases where the agent successfully completed tasks only during specific portions of the training process, with the visible segments corresponding precisely to those successful completion instances.

With an alternating step value of 1, the USV agent demonstrated significantly compromised performance: minimal task completions, high

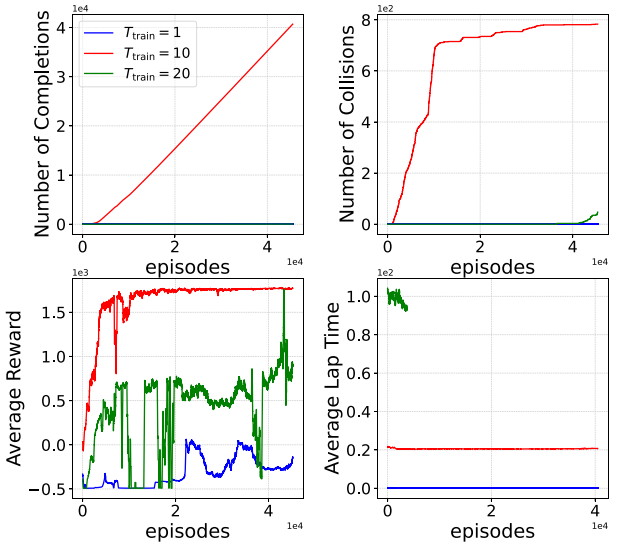


Fig. 16. Parameter sensitivity analysis: Training results of USV on square paths with obstacles in straight sections under different alternating training steps (1, 10, and 20).

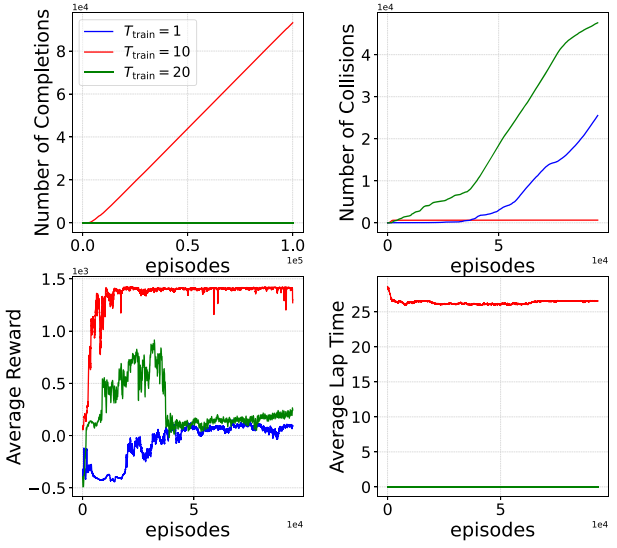


Fig. 17. Parameter sensitivity analysis: Training results of USV on square paths with obstacles in curved sections under different alternating training steps (1, 10, and 20).

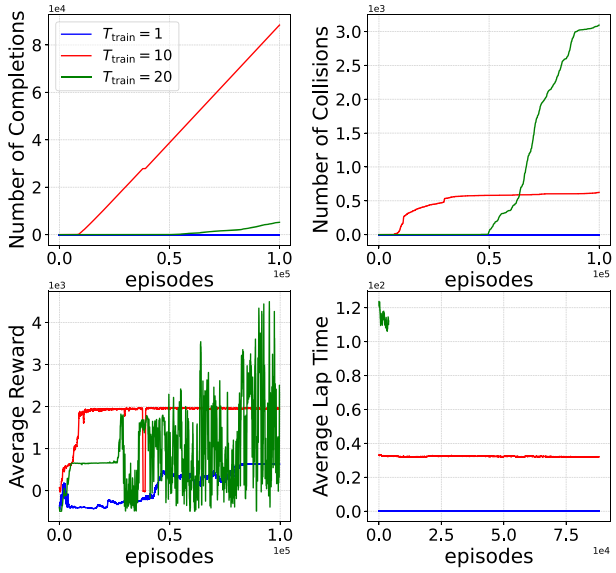


Fig. 18. Parameter sensitivity analysis: Training results of USV on W-shape paths with obstacles in straight sections under different alternating training steps (1, 10, and 20).

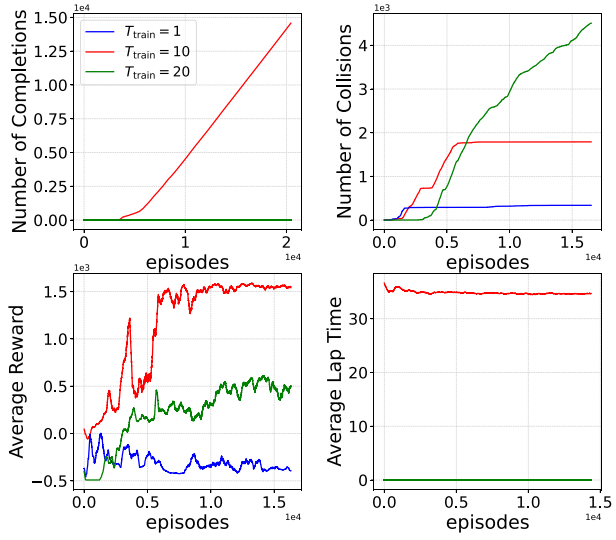


Fig. 19. Parameter sensitivity analysis: Training results of USV on W-shape paths with obstacles in curved sections under different alternating training steps (1, 10, and 20).

collision frequency, consistently low reward curves, and no recorded average completion times due to persistent course failures. When the alternating steps were increased to 20, the USV agent showed notable improvement with higher completion rates, reduced collisions, elevated reward curves, and decreased completion times, although completion time data remained limited due to fewer successful runs.

The optimal performance was achieved with 10 alternating training steps, where the USV agent exhibited maximum completion rates, minimum collision incidents, highest cumulative rewards, and shortest average completion times. These findings clearly indicate that the ADRL algorithm's effectiveness is substantially influenced by the alternating training step parameter.

Further analysis reveals that this performance pattern stems from several key factors: At step value 1, the excessively frequent policy alternation introduces instability and impedes convergence. At step value 10, the moderate alternation frequency provides sufficient optimization time for each policy while maintaining responsiveness to the opponent's strategy. At step value 20, the excessive interval between alternations may result in delayed policy updates, causing the agent's strategy to lag behind the opponent's adaptations, thereby compromising overall effectiveness. These results demonstrate that selecting an appropriate number of alternating training steps is critical for maximizing ADRL algorithm performance in USV navigation scenarios.

Beyond the alternating training steps analysis, a comprehensive hyperparameter sensitivity study was conducted to investigate the broader impact of key ADRL parameters on system performance. This extended analysis provides essential guidance for practitioners implementing ADRL in autonomous navigation applications, complementing the alternating steps findings with insights into other critical algorithmic components.

The comprehensive parameter sensitivity analysis presented in Fig. 20 provides crucial insights into the impact of key hyperparameters

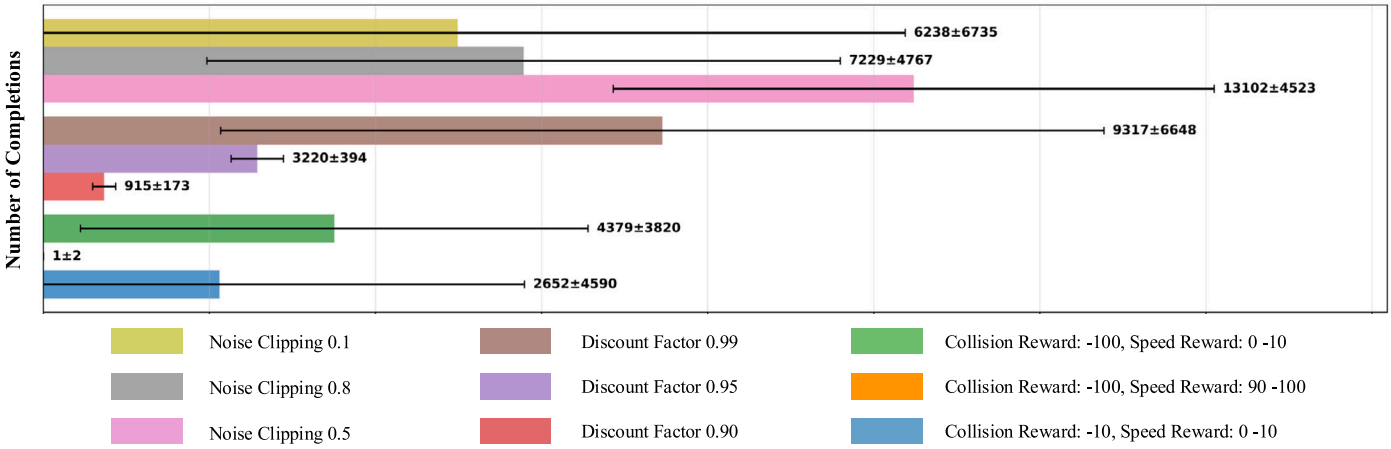


Fig. 20. Parameter sensitivity analysis: Number of completions under different hyperparameter configurations including noise clipping: 0.1, 0.5, 0.8, discount factor: 0.90, 0.95, 0.99, and reward settings: collision penalties and speed reward ranges. Results represent means and standard deviations across 5 independent random seeds.

on ADRL performance beyond the alternating training steps previously examined. To ensure statistical reliability and reproducibility, each parameter configuration was evaluated across 5 independent random seeds, with results reported as means ±standard deviations. This methodical approach confirms that the observed performance differences are statistically significant rather than artifacts of random initialization, providing robust evidence for hyperparameter selection guidelines. The results demonstrate that hyperparameter selection significantly influences training effectiveness, with completion rates varying dramatically across different configurations.

The analysis reveals that noise clipping values have a substantial impact on performance. With noise clipping set to 0.1, the system achieved 6238 ± 6735 completions, showing high variability as indicated by the large standard deviation. Noise clipping of 0.8 resulted in 7229 ± 4767 completions with improved stability. However, the optimal performance was achieved with noise clipping of 0.5, yielding 13102 ± 4523 completions with balanced exploration and stability. This suggests that moderate noise injection facilitates effective exploration while maintaining policy stability, whereas extreme values either limit exploration or introduce excessive randomness that hinders convergence.

The discount factor shows the most pronounced effect on performance among all tested parameters. A discount factor of 0.90 produced only 915 ± 173 completions, indicating that short-sighted policies are inadequate for the complex USV racing task. Increasing the discount factor to 0.95 improved performance to 3220 ± 394 completions, though still suboptimal. The highest performance was achieved with a discount factor of 0.99, resulting in 9317 ± 6648 completions. This demonstrates that long-term planning capability is crucial for USV navigation tasks, where immediate rewards must be balanced against future collision avoidance and successful task completion. The high discount factor enables the agent to consider the long-term consequences of its actions, leading to more strategic and robust navigation policies.

Different reward configurations also significantly impact training outcomes. The collision reward of −100 with speed reward range 0–10 achieved 4379 ± 3820 completions, while the collision reward of −10 with speed reward range 0–10 resulted in 2652 ± 4590 completions. This comparison clearly demonstrates that higher collision penalties are essential for learning safe navigation behaviors. The substantial difference in performance 4379 vs. 2652 completions indicates that insufficient collision penalties lead to policies that prioritize speed over safety, resulting in frequent collisions and task failures. The higher collision penalty −100 effectively shapes the agent's behavior to prioritize collision avoidance, leading to more robust and reliable navigation strategies.

The results suggest complex interactions between different hyperparameters, where optimal performance requires careful balancing of exploration, temporal planning horizon, and behavioral incentives. The ranking of parameter importance based on performance impact is: discount factor > noise clipping > reward configuration. This hierarchy provides practical guidance for hyperparameter tuning in similar adversarial RL applications.

These findings provide valuable insights for practitioners implementing ADRL in autonomous navigation systems, highlighting the critical importance of appropriate hyperparameter selection for achieving robust and effective policies. The analysis demonstrates that while adversarial training introduces additional complexity, proper parameter configuration can lead to significantly improved performance in challenging navigation scenarios.

5. Discussion and future work

This study demonstrates the effectiveness of ADRL for USV racing and obstacle avoidance, but several limitations should be acknowledged. First, our approach relies heavily on simulation environments that may not fully capture the complexity of real-world maritime conditions including water currents, waves, and weather effects. This simulation-reality gap could impact performance when deployed on actual USV platforms.

While our current study demonstrates ADRL's effectiveness in simulation, we acknowledge that the simulation-to-reality gap remains unaddressed and represents a significant limitation. The simulation gap primarily manifests in environmental dynamics, sensor characteristics, and physical constraints. A critical limitation of our current approach is the use of idealized sensors that provide perfect, noise-free measurements with instantaneous response times. In practice, real-world USV sensors experience various forms of degradation that significantly impact navigation performance.

Regarding sensor noise and delay mitigation, future work should implement comprehensive sensor realism strategies. To address these challenges, future research should develop robust sensor fusion frameworks that can maintain navigation performance under degraded sensor conditions. This includes developing redundant sensor configurations with graceful degradation capabilities, and creating sensor health monitoring systems that can detect and compensate for partial sensor failures. Additionally, temporal delays should be modeled using realistic communication protocols with variable latency distributions, while sensor dropout scenarios should be simulated to test system resilience under complete sensor failures.

Future work should focus on several key areas to enhance transferability to real-world scenarios: (1) implementing domain randomization by varying USV physical parameters, environmental conditions (wind forces, water currents, lighting), and sensor noise patterns; (2) employing progressive noise injection strategies through curriculum learning, gradually incorporating measurement noise, sensor failures, and communication delays (10–50 ms variations); (3) enhancing physics-informed simulation by integrating validated marine hydrodynamic models, wave-induced disturbances, and realistic environmental factors; (4) establishing a staged real-world validation framework progressing from controlled pond tests to harbor trials and open-water testing with comprehensive performance metrics.

Additionally, future research should investigate model compression techniques (neural network pruning, quantization, knowledge distillation) to address computational constraints for real-time deployment on embedded USV systems. These future enhancements would significantly strengthen the practical applicability of ADRL-based USV systems.

Second, the adversarial training, while robust, assumes obstacles with specific behavioral patterns; real-world obstacles like other vessels may exhibit more unpredictable behaviors governed by human decision-making. Third, computational requirements for ADRL are significantly higher than traditional methods, potentially limiting real-time implementation on resource-constrained USV hardware.

Quantitative analysis reveals that ADRL's computational overhead is approximately 2.5–3 times that of traditional DRL methods. This increased cost stems from several factors: (1) dual agent architecture requiring separate replay buffers totaling 1.6×10^7 experiences compared to DRL's single 8×10^6 buffer, (2) alternating training cycles that extend convergence time by approximately 40%–60% as shown in training curves (Figs. 8 to 13), and (3) adversarial strategy adaptation requiring additional network update cycles every 10 training steps. During inference, ADRL maintains comparable real-time performance to DRL, as both methods only require forward passes through single actor networks. However, the training phase computational requirements and memory footprint make ADRL approximately 2.8× more resource-intensive than standard DRL approaches, which represents a significant consideration for deployment on embedded USV systems with limited computational capacity.

Future research should address these limitations by incorporating more realistic environmental modeling, expanding the diversity of adversarial behaviors, and optimizing algorithms for deployment on embedded systems. Additionally, integrating safe RL approaches like those proposed in Zhong et al. (2025) could provide formal safety guarantees while maintaining performance. The proposed strategies provide a systematic roadmap for future research to reduce the simulation gap and offer a practical pathway for eventual real-world deployment.

6. Conclusion

In this paper, an approach, autonomous USVs to race with obstacle avoidance based on ADRL is proposed. The ADRL algorithm leverages adversarial training to enhance the robustness and adaptability of USVs in challenging scenarios, such as dynamic racecourses with obstacles. By integrating adversarial obstacles into the training process, ADRL enables USVs to learn effective navigation strategies that prioritize collision avoidance, task completion and time efficiency. The results of our evaluation demonstrate that ADRL outperforms conventional DRL approaches in terms of collision avoidance, task completion rates and time efficiency across various challenging scenarios. ADRL consistently achieves better performance metrics, highlighting its robustness and adaptability in dynamic and adversarial environments. The results of our evaluation suggest that ADRL is a promising approach for enhancing the capabilities of autonomous USVs and improving their navigation in complex and unpredictable environments.

CRediT authorship contribution statement

Jingyu Liu: Writing – original draft, Visualization, Software, Methodology, Conceptualization. **Yuanda Wang:** Writing – review & editing, Project administration, Funding acquisition, Conceptualization. **Changyin Sun:** Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under grants 62136008, in part by the Postdoctoral Fellowship Program of CPSF under grants 2024M750422 and GZB20240144, and in part by the Jiangsu Funding Program for Excellent Postdoctorial Talent under grant 2024ZB047.

Data availability

No data was used for the research described in the article.

References

Bae, Inyeong, Hong, Jungpyo, 2023. Survey on the developments of unmanned marine vehicles: Intelligence and cooperation. Sensors 23 (10), 4643, Multidisciplinary Digital Publishing Institute.

Barrera, Carlos, Padron, I, Luis, F.S, Llinas, Octavio, 2021. Trends and challenges in unmanned surface vehicles (USV): From survey to shipping. TransNav: Int. J. Mar. Navig. Saf. Sea Transp. 15.

Cai, Yunfei, Zheng, Shutao, Liu, Weitian, Qu, Zhiyong, Zhu, Jiyue, Han, Junwei, 2021. Sliding-mode control of ship-mounted Stewart platforms for wave compensation using velocity feedforward. Ocean Eng. 236, 109477, Elsevier.

Chen, Xiao, Liu, Wei, Shuang, Song, 2021. The autonomous path planning algorithm for the USV under complex battlefield situations. In: 2021 40th Chinese Control Conference. CCC, IEEE, pp. 2742–2747.

Chen, Jie, Sun, Jian, Wang, Gang, 2022. From unmanned systems to autonomous intelligent systems. Engineering 12, 16–19, Elsevier.

Chen, Jiaying, Wang, Han, 2020. An obstacle detection method for USV by fusing of radar and motion stereo. In: 2020 IEEE 16th International Conference on Control & Automation. ICCA, IEEE, pp. 159–164.

Cheng, Chen, Liu, Dong, Du, Jin-Hui, Li, Yong-Zheng, 2023. Research on visual perception for coordinated air–sea through a cooperative USV-uav system. J. Mar. Sci. Eng. 11 (10), 1978, MDPI.

Dijkstra, Edsger W, 2022. A note on two problems in connexion with graphs. In: Edsger Wybe Dijkstra: His Life, Work, and Legacy. pp. 287–290.

Fahlstrom, Paul G., Gleason, Thomas J., Sadraey, Mohammad H, 2022. Introduction To UAV Systems. John Wiley & Sons.

Fan, Yunsheng, Sun, Zhe, Wang, Guofeng, 2022. A novel reinforcement learning collision avoidance algorithm for USVs based on maneuvering characteristics and COLREGs. Sensors 22 (6), 2099, MDPI.

Fan, Yunsheng, Sun, Zhe, Wang, Guofeng, 2023. A novel intelligent collision avoidance algorithm based on deep reinforcement learning approach for USV. Ocean Eng. 287, 115649, Elsevier.

Farella, Alessia, Paciolla, Francesco, Quartarella, Tommaso, Pascuzzi, Simone, 2024. Agricultural Unmanned Ground Vehicle (UGV): A Brief Overview, International Symposium on Farm Machinery and Processes Management in Sustainable Agriculture. Springer, pp. 137–146.

Fujimoto, Scott, Hoof, Herke, Meger, David, 2018. Addressing function approximation error in actor-critic methods. In: International Conference on Machine Learning. PMLR, pp. 1587–1596.

Gonzalez-Garcia, Alejandro, Castañeda, Herman, 2021. Guidance and control based on adaptive sliding mode strategy for a USV subject to uncertainties. IEEE J. Ocean. Eng. 46 (4), 1144–1154, IEEE.

Hart, Peter E., Nilsson, Nils J., Raphael, Bertram, 1968. A formal basis for the heuristic determination of minimum cost paths. IEEE Trans. Syst. Sci. Cybern. 4 (2), 100–107, IEEE.

Jiang, Haoge, Wang, Han, Yau, Wei-Yun, Wan, Kong-Wah, 2020. A brief survey: Deep reinforcement learning in mobile robot navigation. In: 2020 15th IEEE Conference on Industrial Electronics and Applications. ICIEA, IEEE, pp. 592–597.

Juliani, Arthur, Berges, Vincent-Pierre, Teng, Ervin, Cohen, Andrew, Harper, Jonathan, Elion, Chris, Goy, Chris, Gao, Yuan, Henry, Hunter, Mattar, Marwan, Lange, Danny, 2020. Unity: A general platform for intelligent agents. arXiv preprint arXiv:1809.02627.

Karaman, Sertac, Walter, Matthew R., Perez, Alejandro, Frazzoli, Emilio, Teller, Seth, 2011. Anytime motion planning using the RRT. In: 2011 IEEE International Conference on Robotics and Automation. IEEE, pp. 1478–1483.

Li, Xiang, Li, Gang, Bian, Zijian, 2024a. Research on autonomous vehicle path planning algorithm based on improved rrt* algorithm and artificial potential field method. Sensors 24 (12), 3899, MDPI.

Li, Zhenglin, Yuan, Tianxin, Ma, Liyan, Zhou, Yang, Peng, Yan, 2024b. Target detection for USVs by Radar–Vision fusion with swag-robust distance-aware probabilistic multimodal data association. IEEE Sens. J. 24 (12), 20177–20187. http://dx.doi.org/10.1109/JSEN.2024.3394703.

Liu, Tao, Dong, Zaopeng, Du, Hongwang, Song, Lifei, Mao, Yunsheng, 2017. Path following control of the underactuated USV based on the improved line-of-sight guidance algorithm. Pol. Marit. Res. 24 (1), 3–11.

Ma, Yong, Zhao, Yujiao, Li, Zhixiong, Yan, Xinping, Bi, Huaxiong, Królczyk, Grzegorz, 2022. A new coverage path planning algorithm for unmanned surface mapping vehicle based on A-star based searching. Appl. Ocean Res. 123, 103163, Elsevier.

Ma, Yong, Zhao, Yujiao, Wang, Yulong, Gan, Langxiong, Zheng, Yuanzhou, 2020. Collision-avoidance under COLREGS for unmanned surface vehicles via deep reinforcement learning. Marit. Policy Manag. 47 (5), 665–686, Taylor & Francis.

Metekia, Enanwo Wondem, Asfaw, Wubshet Ayalew, Abdissa, Chala Merga, Lemma, Lebsework Negash, 2025. Control of a fixed wing unmanned aerial vehicle using a robust fractional order controller. Sci. Rep. 15 (1), 19954, Nature Publishing Group UK London.

Mohammed, Tofik Kemal, Asfaw, Wubshet Ayalew, Abdissa, Chala Merga, Lemma, Lebsework Negash, 2025. Constrained robust adaptive control design for fixed wing UAV under parameter uncertainties and external disturbances. Eng. Res. Express 7 (2), 025254, IOP Publishing.

Mohanty, Prases K., Singh, Anand Kumar, Kumar, Amit, Mahto, Manjeet Kumar, Kundu, Shubhasri, 2021. Path planning techniques for mobile robots: A review. In: International Conference on Soft Computing and Pattern Recognition. Springer, pp. 657–667.

Noreen, Iram, Khan, Amna, Habib, Zulfiqar, 2016. Optimal path planning using RRT* based approaches: A survey and future directions. Int. J. Adv. Comput. Sci. Appl. 7 (11), Science and Information (SAI) Organization Limited.

Pan, Zhenhua, Li, Dongfang, Yang, Kun, Deng, Hongbin, 2019. Multi-robot obstacle avoidance based on the improved artificial potential field and PID adaptive tracking control algorithm. Robotica 37 (11), 1883–1903, Cambridge University Press.

Shimizu, Shoma, Nishihara, Kenta, Miyauchi, Yoshiki, Wakita, Kouki, Suyama, Rin, Maki, Atsuo, Shirakawa, Shinichi, 2022. Automatic berthing using supervised learning and reinforcement learning. Ocean Eng. 265, 112553.

Sun, Shuai, Lyu, Hongguang, Gao, Zongjiang, Yang, Xiao, 2024. Grid map assisted radar target tracking in a detection occluded maritime environment. IEEE Trans. Instrum. Meas. 73, 1–11. http://dx.doi.org/10.1109/TIM.2024.3381495.

Tan, Kai Liang, Esfandiari, Yasaman, Lee, Xian Yeow, Sarkar, Soumik, et al., 2020. Robustifying reinforcement learning agents via action space adversarial training. In: 2020 American Control Conference. ACC, IEEE, pp. 3959–3964.

Villa, Jose, Aaltonen, Jussi, Koskinen, Kari T., 2020. Path-following with lidar-based obstacle avoidance of an unmanned surface vehicle in harbor conditions. IEEE/ASME Trans. Mechatronics 25 (4), 1812–1820, IEEE.

Wang, Yuanda, Cao, Jingyu, Sun, Jia, Zou, Xuesong, Sun, Changyin, 2024a. Path following control for unmanned surface vehicles: A reinforcement learning-based method with experimental validation. IEEE Trans. Neural Netw. Learn. Syst. 35 (12), 18237–18250. http://dx.doi.org/10.1109/TNNLS.2023.3313312.

Wang, Di, Chen, Haiming, Lao, Sihan, Drew, Steve, 2023. Efficient path planning and dynamic obstacle avoidance in edge for safe navigation of USV. IEEE Internet Things J..

Wang, Jia, Wang, Rongtao, Lu, Daohua, Zhou, Hao, Tao, Tianyi, 2022. USV dynamic accurate obstacle avoidance based on improved velocity obstacle method. Electronics 11 (17), 2720, MDPI.

Wang, Yuanda, Yuan, Xin, Sun, Changyin, 2024b. Learning autonomous race driving with action mapping reinforcement learning. ISA Trans. 150, 1–14. http://dx.doi.org/10.1016/j.isatra.2024.05.010, Elsevier.

Wu, Xing, Chen, Haolei, Chen, Changgu, Zhong, Mingyu, Xie, Shaorong, Guo, Yike, Fujita, Hamido, 2020. The autonomous navigation and obstacle avoidance for USVs with ANOA deep reinforcement learning method. Knowl.-Based Syst. 196, 105201, Elsevier.

Wu, Huafeng, Zhang, Kun, Mei, Xiaojun, Liang, Linian, Zhang, Zhiheng, Wang, Feng, Han, Bing, Han, Dezhi, Li, Kuan-Ching, 2025. Heuristic strategy-empowered real-time path following for autonomous surface vessel with adaptive line-of-sight guidance. IEEE J. Ocean. Eng. 50 (1), 307–323. http://dx.doi.org/10.1109/JOE.2024.3447877.

Xiao, Chen, Zhong, Liu, Zhang, Jianqiang, Zhou, Dechao, Jiao, Dong, 2018. Adaptive sliding-mode path following control system of the underactuated USV under the influence of ocean currents. J. Syst. Eng. Electron. 29 (6), 1271–1283, BIAI.

Xie, Wei, Reis, Joel, Cabecinhas, David, Silvestre, Carlos, 2020. Design and experimental validation of a nonlinear controller for underactuated surface vessels. Nonlinear Dynam. 102, 2563–2581, Springer.

Yan, Zheping, Gong, Peng, Zhang, Wei, Wu, Wenhua, 2020. Model predictive control of autonomous underwater vehicles for trajectory tracking with external disturbances. Ocean Eng. 217, 107884, Elsevier.

Yan, Nan, Huang, Subin, Kong, Chao, 2021. Reinforcement learning-based autonomous navigation and obstacle avoidance for USVs under partially observable conditions. Math. Probl. Eng. 2021 (1), 5519033, Wiley Online Library.

Zhao, Baigang, Zhang, Xianku, Liang, Cailei, Han, Xu, 2021. An improved model predictive control for path-following of USV based on global course constraint and event-triggered mechanism. IEEE Access 9, 79725–79734, IEEE.

Zheng, Baojin, Zhu, Ming, Guo, Xiao, Ou, Jiajun, Yuan, Jiace, 2024. Path planning of stratospheric airship in dynamic wind field based on deep reinforcement learning. Aerosp. Sci. Technol. 150, 109173, Elsevier.

Zhong, Yichao, Zhang, Chong, He, Tairan, Shi, Guanya, 2025. Bridging adaptivity and safety: Learning agile collision-free locomotion across varied physics. arXiv preprint arXiv:2501.04276.

Zhou, Guoqing, Wu, Jinhuang, Gao, Ke, Song, Naihui, Jia, Guoshuai, Zhou, Xiang, Xu, Jiasheng, Wang, Xia, 2024. Development of an adaptive fuzzy integral-derivative line-of-sight method for bathymetric LiDAR onboard unmanned surface vessel. Remote. Sens. 16 (14), 2657, MDPI.

Zhu, Zhenyu, Li, Lianbo, Wu, Wenhao, Jiao, Yang, 2021. Application of improved dijkstra algorithm in intelligent ship path planning. In: 2021 33rd Chinese Control and Decision Conference. CCDC, IEEE, pp. 4926–4931.

Zissis, Dimitris, Troupiotis-Kapeliaris, Alexandros, Xidias, Elias, Kogias, Thomas, Spiliopoulos, Giannis, Millefiori, Leonardo, Braca, Paolo, 2024. The aegean ro-boat race: A regatta to accelerate innovation in marine robotics and autonomous vessels. IEEE Aerosp. Electron. Syst. Mag. 39 (9), 36–45. http://dx.doi.org/10.1109/MAES.2024.3425317, IEEE.